Лабораторная работа № 7.
Работа с БД с использованием JDBC

Цель работы

Получить общее представление о прикладном программном интерфейсе JDBC для работы с БД. Освоить процедуру установки соединения с БД.

1. Теоретические сведения

1.1 Общие сведения о JDBC

JDBC (Java Database Connectivity) – **прикладной программный интерфейс** (application programming interface - API) для доступа к реляционным базам данных. JDBC позволяет выполнять соединение с базой данных, посылать запросы на получение и изменение данных, а также выполнять манипуляции с полученными данными. С технической точки зрения API JDBC представляет набор классов в библиотеке java.sql. Для работы с конкретной СУБД через JDBC необходим соответствующий драйвер.

JDBC - это «родной» для Java интерфейс к базовым абстракциям и концепциям языка SQL Использование JDBC позволяет разрабатывать переносимые приложения на Java, не зависящие от конкретной СУБД.

JDBC включает компоненты:

1. **Прикладной интерфейс** JDBC (JDBC API) — обеспечивает доступ к реляционным данным для выполнения перечисленных выше функций. Прикладной интерфейс является частью платформ Java Standard Edition (Java SE) и Java Enterprise Edition (Java EE) и реализуется в виде двух интерфейсов: основного (пакет java.sql) и расширенного (пакет javax.sql). Конкретный набор классов, реализующий JDBC-интерфейс и осуществляющий доступ к конкретной СУБД, называется **драйвером**. JDBC-драйверы для своих СУБД поставляют все основные разработчики.

Все основные сущности в JDBC API, которые используются для работы с БД, являются интерфейсами: Connection, Statement, PreparedStatement, CallableStatement, ResultSet.

JDBC драйвер конкретной базы данных предоставляет реализации этих интерфейсов.

2. **Менеджер драйверов** JDBC (JDBC Driver Manager) — небольшой и простой класс, обеспечивающий присоединение Java-приложений к требуемому драйверу JDBC.

Для установки драйвера JDBC, как правило, необходимо скопировать библиотеку (пакет) драйвера на компьютер, а затем добавить путь к библиотеке в пути classpath (или через свойства проекта в Eclipse).

Рассмотрим использование JDBC на примере работы с СУБД

PostgreSQL. Рекомендуется устанавливать версии PostgreSQL 9.5 или 9.6. Информация по другим СУБД приводится в примечаниях далее по тексту.

1.2 Подготовка для работы с БД

Для работы с БД необходимо использовать один из двух вариантов:

1) на компьютер установить соответствующую СУБД (в этом случае сервер БД будет на локальном компьютере)

2) если сервер БД находится на сетевом компьютере, то на локальном компьютере надо установить клиентскую часть СУБД.

Для локальной СУБД на компьютере будет создан сервис (служба), который автоматически будет запускать сервер БД при старте операционной системы.

Для Maven проекта с API JavaFX драйвер подключается через зависимости в файле pom.xml (см. Приложение 1)

Примечание. Для других СУБД есть особенности при выполнении установки соединения, которые описаны в приложении 4.

1.3 Соединение с БД

Для работы с БД приложению необходимо установить соединение с сервером БД. В ходе сессии соединения приложения посылает на сервер SQL-запросы и получает результаты (наборы данных). Для установления соединением и работы с БД используется несколько классов: `DriverManager`,`Connection`.

Класс `DriverManager` пытается по заданным параметрам найти драйвер JDBC, который используется для соединения с БД, и открыть соединение для выполнения запросов к БД.

Предварительно необходимо загрузить класс драйвера вызовом метода

```
Class.forName (имя драйвера)
```

который регистрирует драйвер в классе `DriverManager`.

Для СУБД PostgreSQL имя драйвера (с указанием пути к классу):

```
org.postgresql.Driver
```

Примечание. С появлением JDBC 4.0 необходимость в регистрации драйвера отпадает, как экземпляр драйвера загружается автоматически при попытке получения соединения с БД.

Класс `Connection` используется для соединения с БД, а его методы обеспечивают выполнение запросов к БД .

Стандартный способ получения соединения - это вызов статического метода `getConnection` класса `DriverManager`:

```
DriverManager.getConnection(url, user, pwd)
```

где user - логин, pwd - пароль, url - строка, по которой JDBC определяет параметры БД, с которой устанавливается соединение.
Строка url имеет следующий формат:

<протокол:>[<имя хоста:порт>]<имя экземпляра БД>[?<схема>]

где

протокол – всегда jdbc. Для него может быть задан подпротокол – это тип БД. Для работы с БД Postgres - это postgresql.

имя хоста – это IP-адрес или dns-имя сервера БД (для локального сервера – это localhost).

порт - для Postgres - 5432

Пример строки соединения (имя БД postgres, а имя схемы prod)

а) локальный доступ

jdbc:postgresql://localhost:5432/postgres?currentSchema=prod

б)сетевой доступ

jdbc:postgresql://asoiumn1.asoiu:5432/postgres?currentSchema=prod

Имя схемы в строке соединения можно указывать для обновленного драйвера (начиная с версии JDBC версии 9.4).

1.4 Выполнение запросов

После того как установлено соединение, оно используется для выполнения SQL-запросов к БД.

В JDBC есть три класса для посылки SQL-запросов в БД и три метода в интерфейсе Connection, которые создают экземпляры этих классов. Эти классы и методы, которые их создают, перечислены ниже:

1) Объект класса Statement - создается методом createStatement класса Connection. Объект данного класса используется для выполнения простых SQL-запросов.

2) Объект класса PreparedStatement - создается методом prepareStatement класса Connection. Объект данного класса используется в SQL-запросах с одним или более входными параметрами. Класс PreparedStatement содержит группу методов, устанавливающих значения входных параметров, которые отсылаются в БД при выполнении запроса. Объект класса PreparedStatement потенциально может быть более эффективным, чем объект класса Statement, так как он предварительно компилируется и сохраняется для будущего использования.

3) Объект класса CallableStatement - создается методом prepareCall класса Connection. Объекты CallableStatement используются для выполнения хранимых процедур - именованных процедур, хранящихся в БД. Объект класса CallableStatement позволяет

использовать не только входных параметров, но и принимать значения выходных параметров для получения результатов.

Пример

```
Connection con = this.getConnection();
// Формирование запроса к БД
CallableStatement statement = con.prepareCall(
      " { call lot.update_stat_akc() } ");
statement.execute();
```

1.5 Последовательность действий для работы с СУБД PostgreSQL

Для того чтобы подключиться к СУБД и выполнить запрос, необходимо написать код для выполнения ***следующих действий***:

1. Подключение необходимых библиотек (пакетов) директивой `import`
2. Регистрация JDBC драйвера
3. Установка соединения с БД
4. Создание объекта запроса
5. Выполнение запроса и получение результирующего набора (при необходимости). В случае получения набора данных выполняются шаги 6-8
6. Обработка полученного набора
7. Закрытие набора и объекта запроса
8. Закрытие соединения

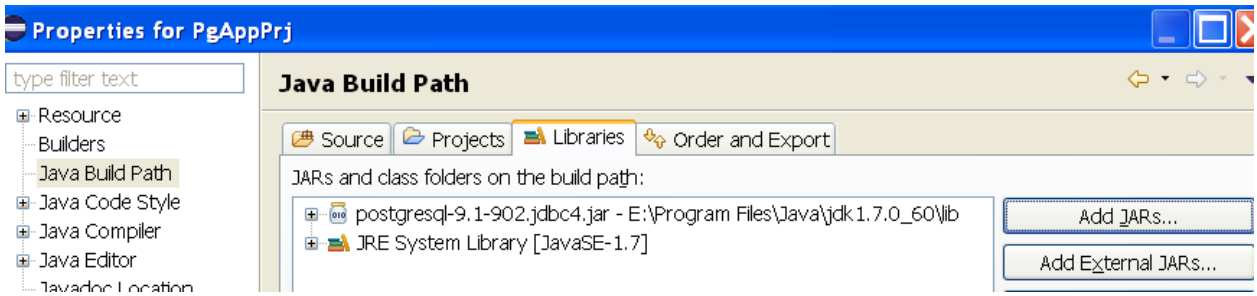
Рассмотрим выполнение указанных действий более подробно.

Шаг 1. Подключение необходимых библиотек

Необходимо подключить стандартный пакет JDBC, используя директиву:

```
import java.sql.*;
```

Для подключения драйвера СУБД PostgreSQL необходимо указать положение драйвера JDBC в переменной `classpath`. А в среде разработки Eclipse подключение можно выполнить в настройках проекта. В разделе Java Build Path – Libraries нажать кнопку Add External JARs и найти папку с драйвером. В проекте Maven драйвер добавляется автоматически, путем указания зависимости в файле pom.xml.



Шаг 2. Установка (открытие) соединения

Необходимо создать соединение вызовом метода `getConnection()` класса `DriverManager`, возвращающим объект типа Connection.

```
Connection conn = DriverManager.getConnection(
              строка_соединения, пользователь, пароль)
```

Программный код

```
Connection con =
DriverManager.getConnection(url, log, pwd);
```

В этом коде пользователь (log) и пароль (pwd) задаются в окне диалога регистрации приложения БД, а строка соединения (url) может быть задана в приложении или считана из файла.

Для БД PostgreSQL строка соединения:

```
jdbc:postgresql://localhost:5432/имя_БД.
```

Шаг 3. Создание объекта запроса

Для создания объекта `Statement`, включающего текст оператора запроса, используется метод `createStatement` класса соединения

```
Statement stmt = con.createStatement();
```

Шаг 4. Выполнение запроса

Для выполнения запроса к БД используется метод `executeQuery` класса `Statement`. Полученный результат возвращается в виде объекта класса `ResultSet`.

Пример. Выбор данных из таблицы БД `Sprod`.

```
ResultSet rset = stmt.executeQuery ("SELECT * FROM Sprod");
```

Шаг 5. Обработка набора данных

Объект `ResultSet` поддерживает курсор - указатель на текущую строку набора данных. При получении набора данных курсор позиционируется перед первой строкой набора. Для изменения положения курсора используется метод набора данных `next()`, который перемещает курсор на следующую запись. Если записи исчерпаны, то метод возвращает значение `false`. Просмотр строк полученных данных выполняется в цикле до тех пор, пока не будет достигнут конец набора данных. Для получения значения колонки текущей строки используется метод `getXXX()` класса `ResultSet`, где XXX – предопределенный тип Java. Нумерация колонок начинается с 1.

Пример. Считывание из каждой строки полученного набора данных первой колонки символьного типа.

```
while (rset.next())
    System.out.println(rset.getString(1));
```

Шаг 6. Закрытие набора данных и оператора запроса

После обработки данных необходимо явно закрывать экземпляры типов `Statement и ResultSet` вызовом метода `close()`.

```
rset.close();
stmt.close();
```

Замечание. Более правильно проверять существование объекта перед вызовом оператора закрытия для исключения ситуации получения ошибки `NullPointerException`

```
if (rset != null) {
   rset.close();
}
if (stmt != null) {
   stmt.close();
}
```

Шаг 7. Закрытие соединение

После завершения работы необходимо закрыть соединение

```
conn.close()
```

Пример. Соединение с СУБД и выполнение запроса на получение номера версии

```
import java.sql.*;
public class TestConnection {
  public static void main(String[] args) {
    Connection con = null;
    Statement stmt = null;
    ResultSet res = null;
    String url =
       "jdbc:postgresql://localhost/postgres";
    String user = "postgres";
    String password = "postgres";
  try {
   con = DriverManager.getConnection(url,user,password);
    stmt = con.createStatement();
    res = stmt.executeQuery("SELECT VERSION()");
    if (res.next()) {
         System.out.println(res.getString(1));
    }
        } catch (SQLException ex) {
        } finally {
            try {
                if (res != null) {
                    res.close();
                }
                if (stmt != null) {
                    stmt.close();
```

```
                }
                if (con != null) {
                    con.close();
                }
        } catch (SQLException ex) {
            }
        }
    }
}
```

Результат

```
Console
<terminated> TestConn [Java Application] E:\Program Files\Java\jre7\bin\javaw.exe (24 апр. 201
PostgreSQL 9.2.3, compiled by Visual C++ build 1600, 32-bit
```

1.6 Создание простого приложения БД. Использование формы соединения

Обычно для соединения с БД используется диалоговое окно регистрации в приложении БД, в котором вводится имя пользователя и пароль.

Рассмотрение создания простого в приложения, в котором для соединения с БД используется окно диалога приведено в приложении 1.

Соединение с другими СУБД описано в приложении 4.

2. Задание

Изучить предлагаемый теоретический материал.

Создать приложение взаимодействия с БД, включающее окно авторизации и основное окно.

Протестировать работу приложения:

- выполнить из среды разработки;
- выполнить с использованием исполняемого jar-файла.

3. Содержание отчета

1) Краткое описание предметной области
2) Схема логической модели базы данных
3) Текст программы
4) Скриншоты окна авторизации и основного окна программы

Примечание. Каждый пункт отчета начинайте с заголовка

1. Описание предметной области
2. Логическая модель БД

и т.д.

4. Литература

1. Прохоренок Н.А. JavaFX. – СПб: ПХВ-Петербург, 2020
2. Учебное пособие по JavaFX .- https://www.geeksforgeeks.org/javafx-tutorial/
3. Информационные материалы. Доступ к базе данных JDBC. - https://docs.oracle.com/javase/tutorial/jdbc/.